Experience with OpenFresco: Reinforced Concrete Bridge

Vesna Terzic

vesna@berkeley.edu

Department of Civil and Environmental Engineering
University of California, Berkeley



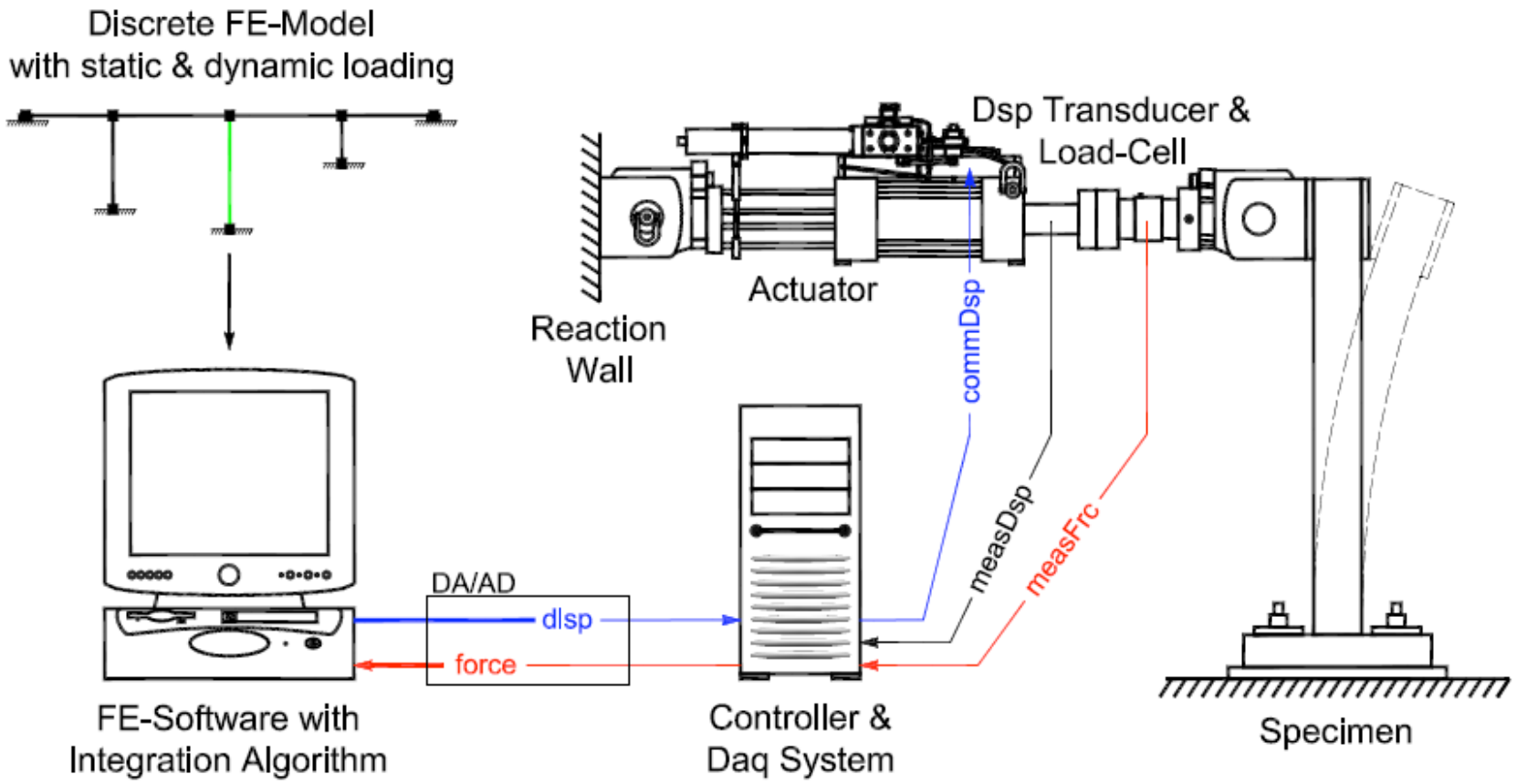
Objective

- Can a damaged bridge carry a heavy truck after an earthquake?
- It is very difficult to physically model the bridge with the (moving) truck on it

  → hybrid simulation is the way to go



Components of a Hybrid Simulation

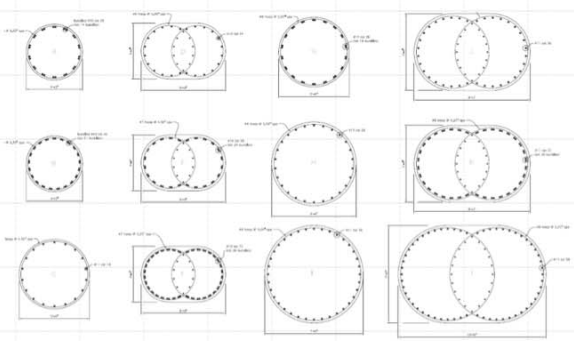


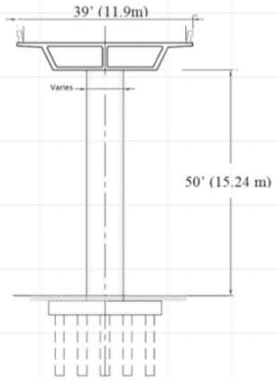
Schellenberg, 2008

Prototype Bridge

- Five span, single column bent overpass bridge



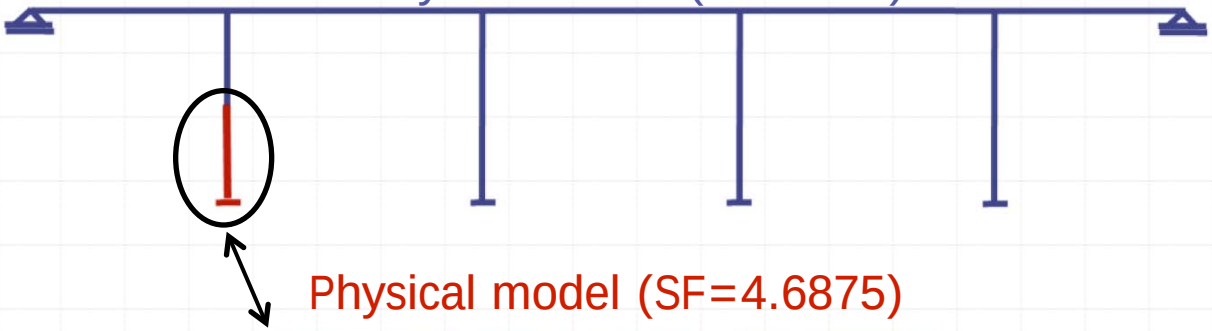




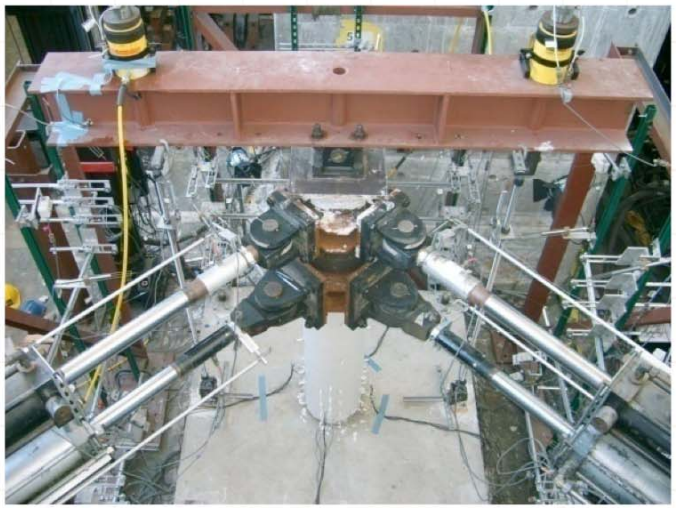
Ketchum et. al., 2004

Hybrid Model of the Bridge

Analytical model (full scale)

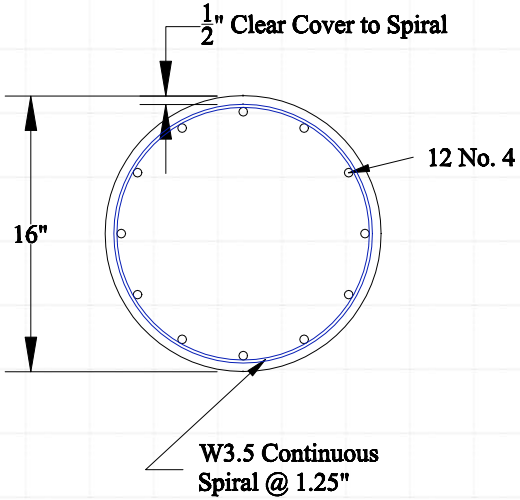


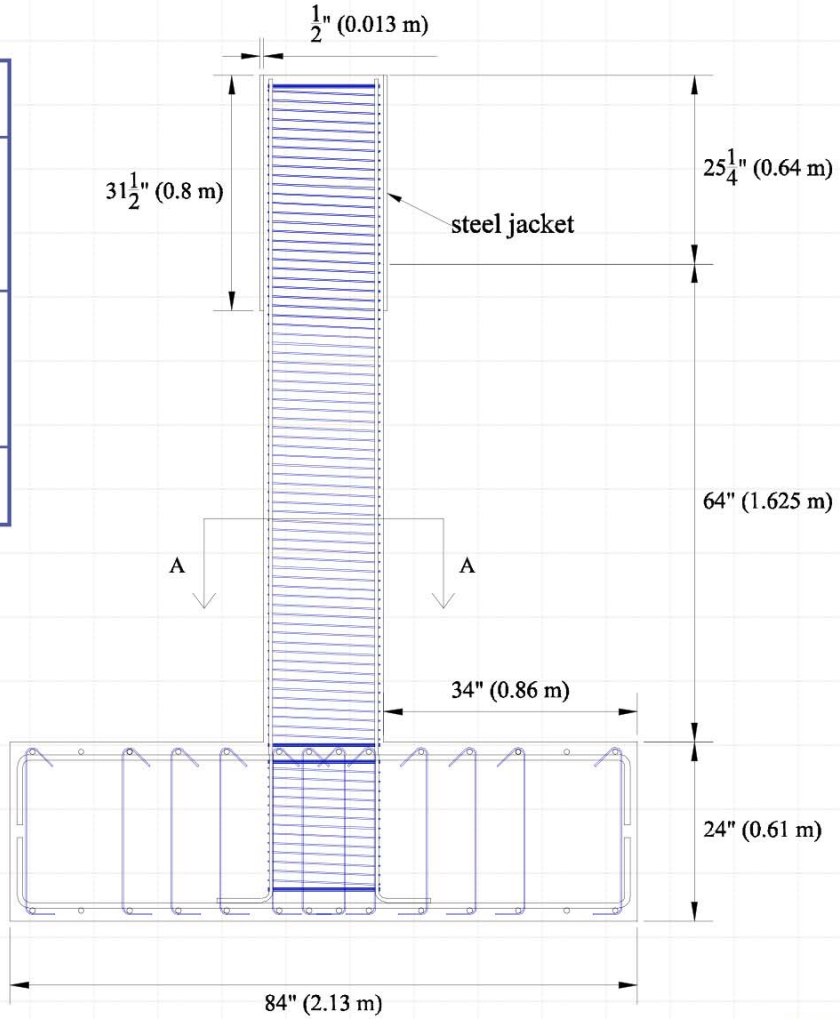
Physical model (SF=4.6875)



Geometry of the Specimen

Diameter	16” (40 cm)
Longitudinal reinforcement	12#4 Gr60 (ρ_l=1.2%)
Transverse reinforcement	W3.5@1.25 in (ρ_t=0.75%)
Concrete	f_c=5 ksi (34.5 MPa)





Calibration of the Analytical Model

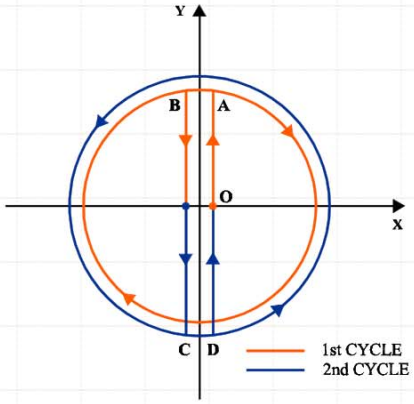
- Bridge columns – nonlinear elements
- Damage is concentrated at plastic hinges
- To calibrate analytical model (used in HS) bi-lateral quasi-static test was performed
- Identical specimens

Quasi-Static Test

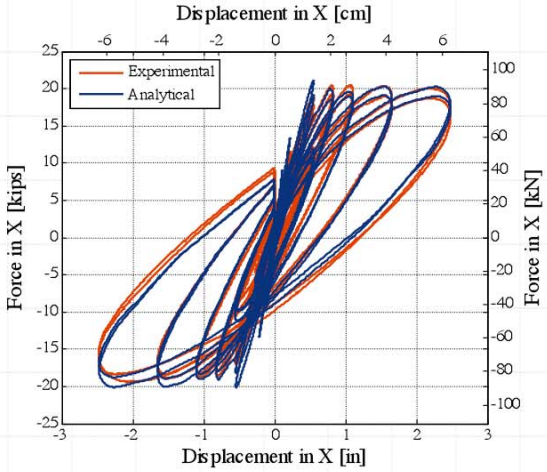
Experimental Setup



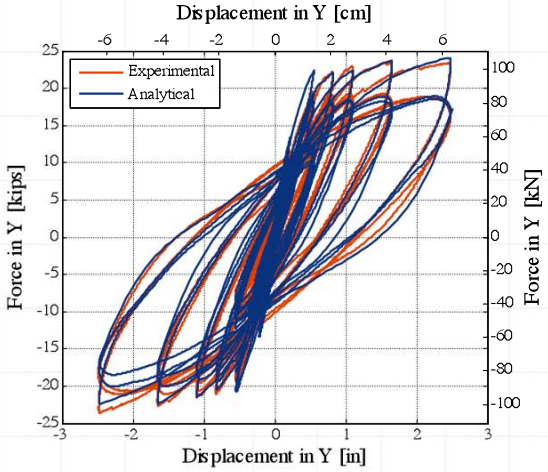
Load Pattern



Material Calibration
Lateral Hystereses



X direction



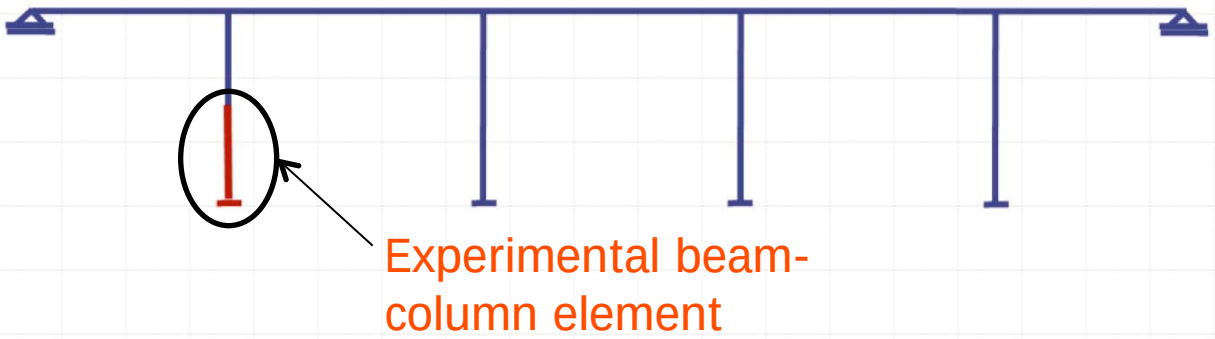
Y direction

Analytical Portion of the Hybrid Model



- Modeled in OpenSees (3D model)
- Deck: linear elastic elements
- Columns: force based nonlinear beam-column elements (calibrated based on quasi-static test)
- Abutments: rollers in both directions
- Integration scheme: Alpha-OS (α=0.9)

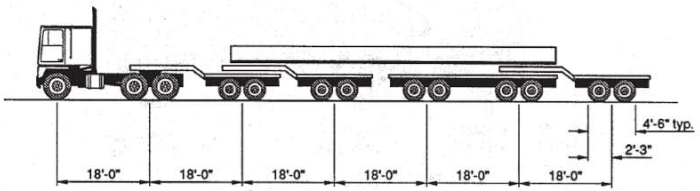
Physical Portion of the Hybrid Model

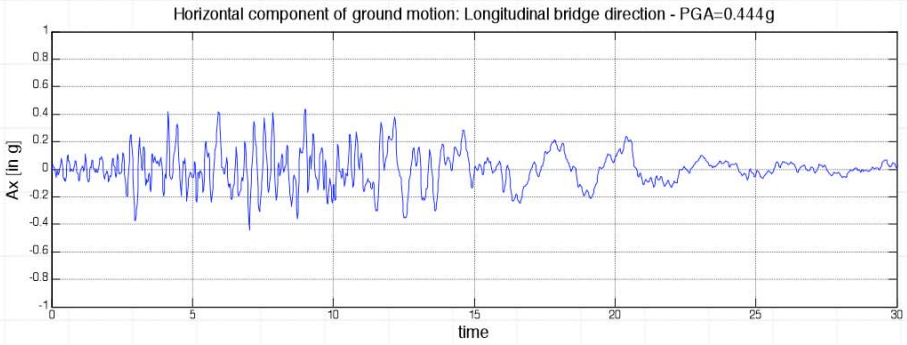


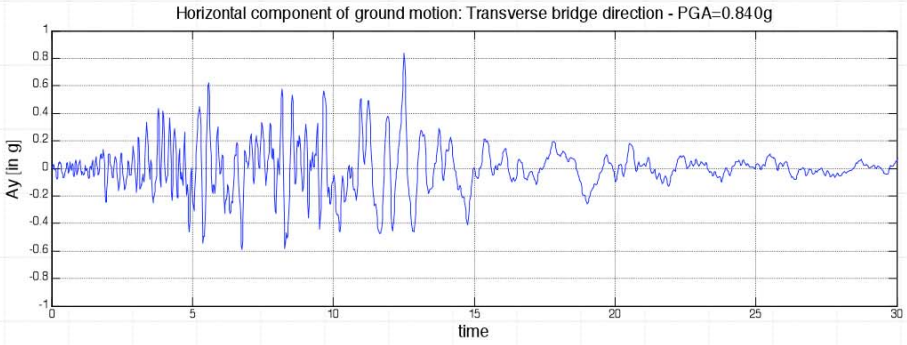
- Scaled model (SF=4.6875)
- OpenFresco was used for hybrid simulation

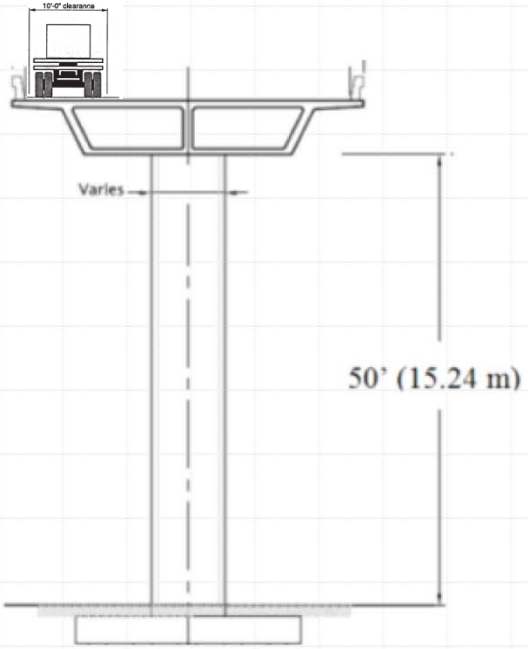
Loads Applied on the Bridge

- Gravity load
- Earthquake load in X and Y direction
- Truck load



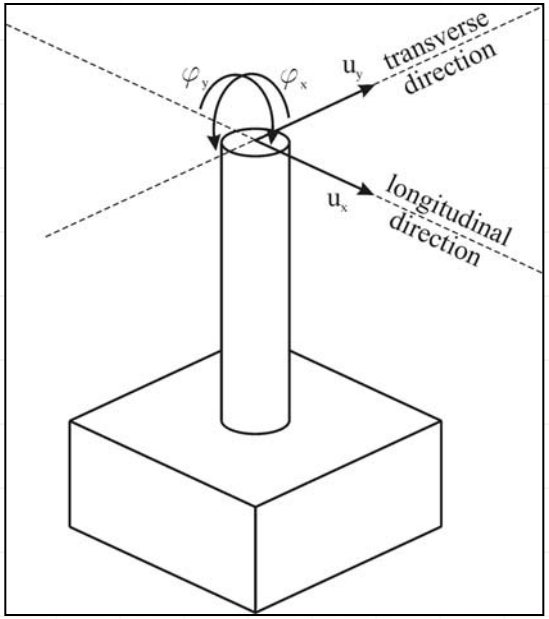






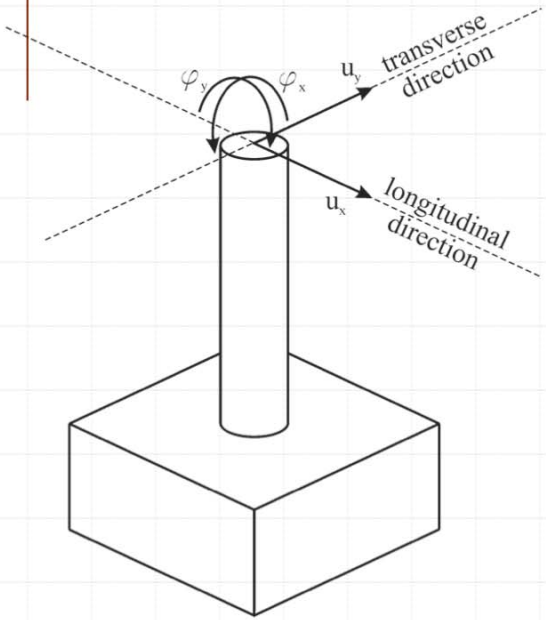
What are we going to control?

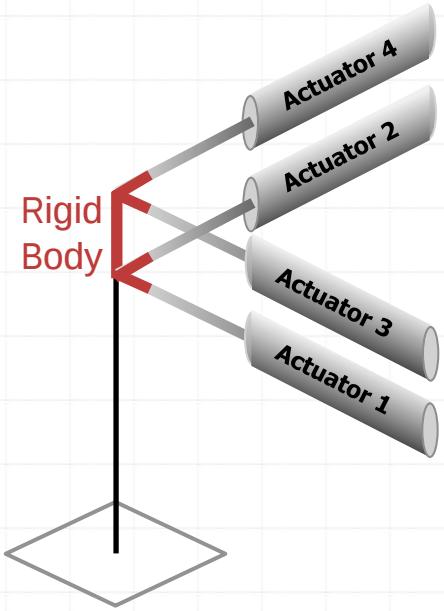
- There are 6 DOFs that could be controlled at the top node of the column:
  - u_x , u_y , u_z , φ_x , φ_y , φ_z
- We are controlling 4 DOFs:
  - u_x , u_y , φ_x , φ_y
  - Assuming that rotation (φ_z) and displacement (u_z) are small enough that they can be neglected.

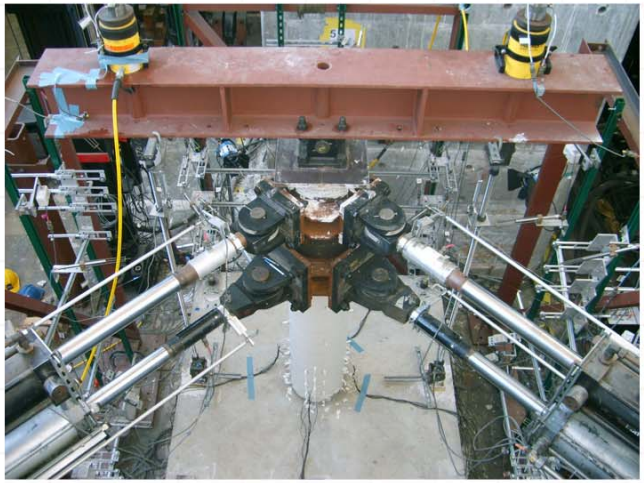


ExpSetup: Lateral Displacements

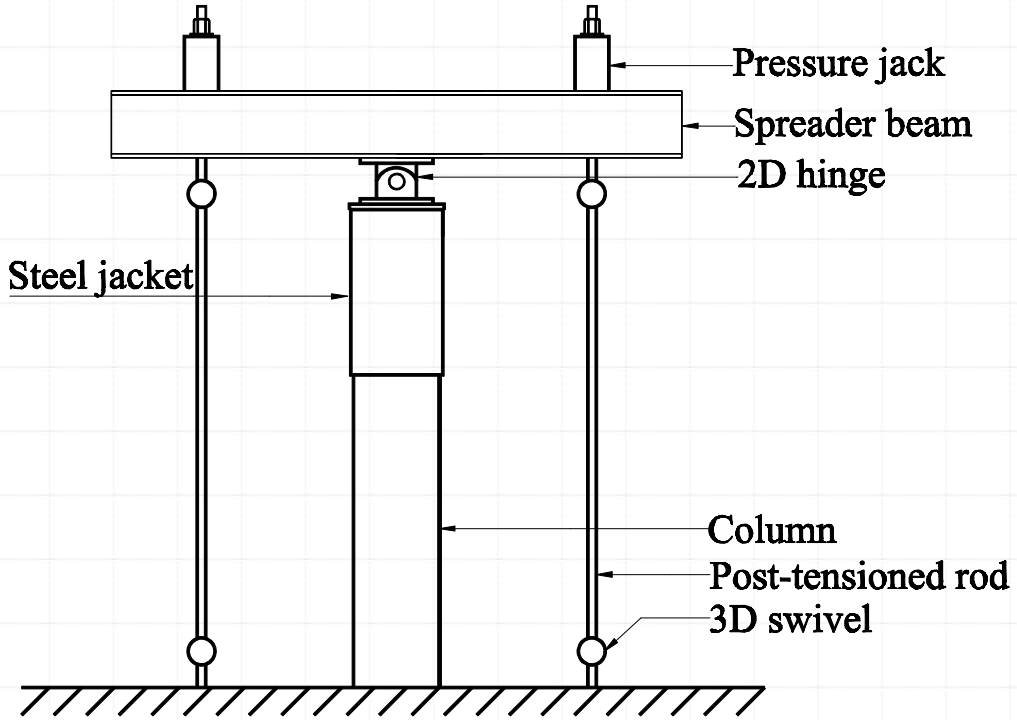
- To control the 4 chosen DOFs we will use 4 actuators:





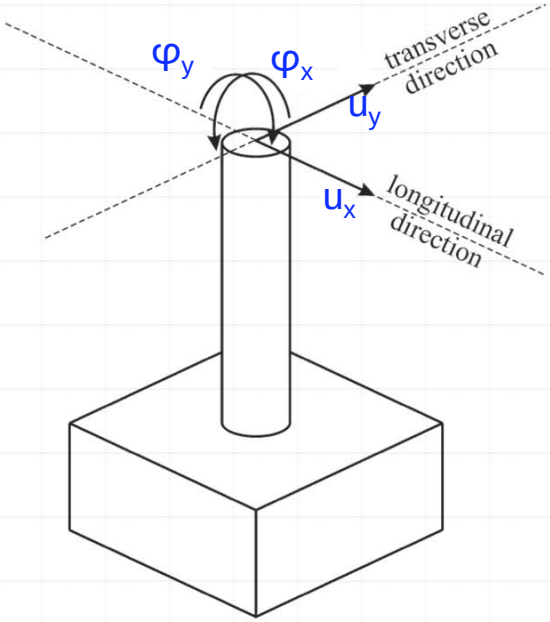


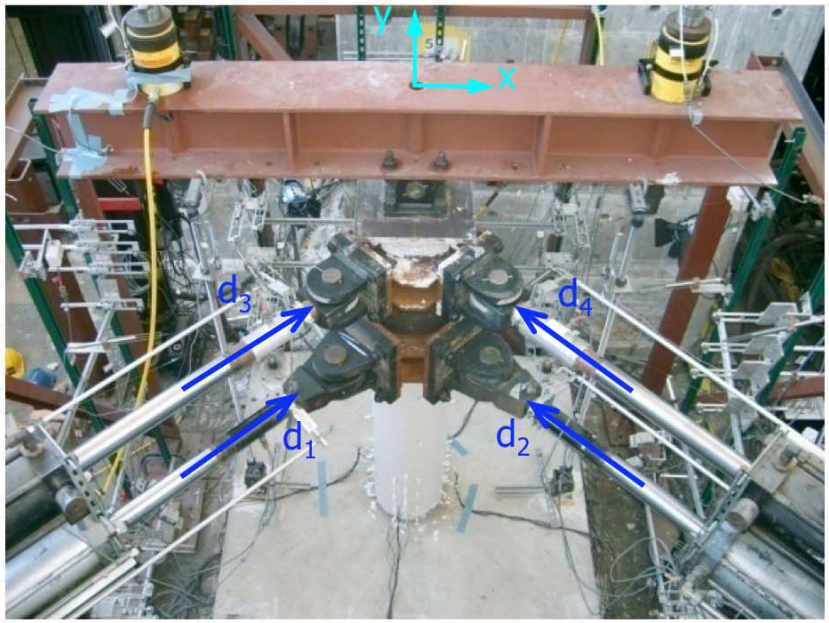
ExpSetup: Axial Load





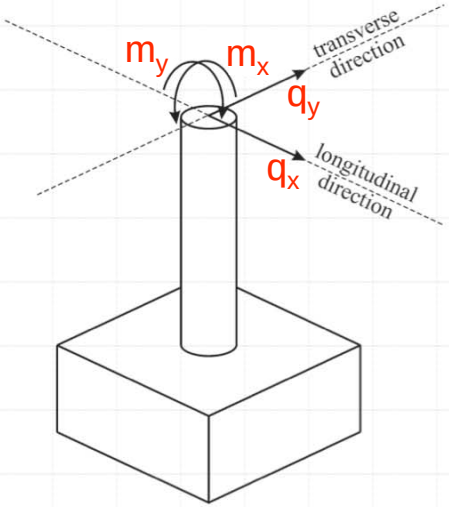
Geometric Transformations: Displacements

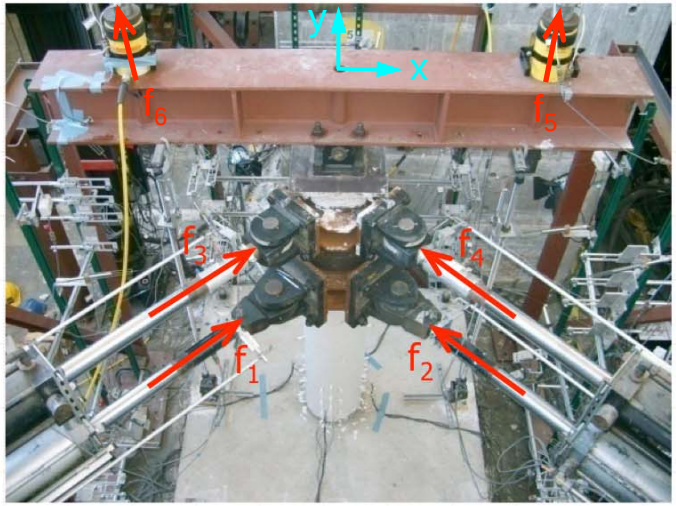




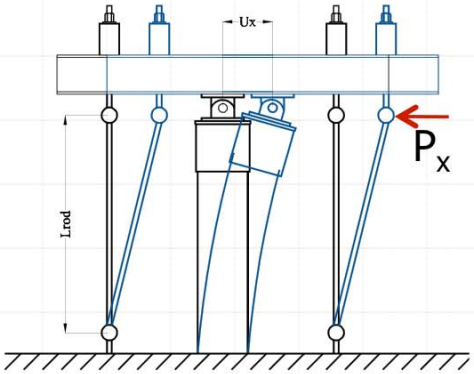
Nonlinear Transformation in ESFourActuators3d

Geometric Transformations: Forces

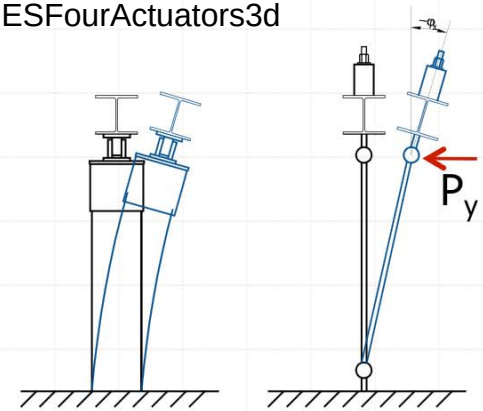




Nonlinear Transformation in ESFourActuators3d



X direction



Y direction

Note: Axial force will also create moments at CP

Deformed configuration
Initial configuration

OpenFresco Interaction with OpenSees

- In FE model (OpenSees framework):
  - loadPackage OpenFresco
  - Define experimental control
  - Define experimental setup
  - Define experimental site
  - Define experimental element

OpenFresco Interaction with OpenSees

- In FE model (OpenSees framework):
  - loadPackage OpenFresco
  - Define experimental control
  - Define experimental setup
  - Define experimental site
  - Define experimental element

OpenFresco Experimental Setup

- ESElementName.h
- ESElementName.cpp
- TclExpSetupCommand.cpp

OpenFresco: ESFourActuators3D.h

```
class ESFourActuators3d : public ExperimentalSetup
{
public:

    // constructors
    ESFourActuators3d(int tag,
        double actLength1, double actLength2, double actLength3, double actLength4,
        double rigidLength1, double rigidLength2, double rigidLength3, double rigidLength4,
        double rigidHeight, double rigidHeight1, double rigidHeight2,
        double rigidLinkN, double rigidLinkS, double rodLengthN, double rodLengthS,
        ExperimentalControl* control = 0,
        int nlGeom = 0, double phiLocX = 45.0);
```

OpenFresco: ESFourActuators3D.h

```
class ESFourActuators3d
(
public:

    // constructors
    ESFourActuators3d(ir
        double actLength
        double rigidLeng
        double rigidHeig
        double rigidLink
        ExperimentalCont
        int nlGeom = 0,
    ESFourActuators3d(cc
```

```
private:
    double L1;          // length of actuator 1
    double L2;          // length of actuator 2
    double L3;          // length of actuator 3
    double L4;          // length of actuator 4
    double a1;           // rigid link length 1
    double a2;           // rigid link length 2
    double a3;           // rigid link length 3
    double a4;           // rigid link length 4
    double h;            // rigid link height between actuators
    double h1;           // rigid link height between lower actuator and pin connection
    double h2;           // rigid link height between lower actuator and lower beam flange
    double arlN;         // rigid link at NORTH rod - top pin to lower beam flange
    double arlS;         // rigid link at SOUTH rod - top pin to lower beam flange
    double LrodN;        // pin to pin length of the north side rod
    double LrodS;        // pin to pin length of the south side rod
    int nlGeom;          // non-linear geometry (0: linear, 1: nonlinear)
    double phiLocX;      // angle of actuator 1 w.r.t reaction wall [deg]

    double uxbeam;
    double uybeam;
    double upN;
    double upS;
    double theta1;
    double theta2;
    double theta3;
    double theta4;
    double theta11;
    double theta22;
    double theta33;
    double theta44;

    Matrix rotLocX;      // rotation matrix
    Vector D;            // displacement vector in x,y axis
    Vector d;            // displacement vector in 1,2 axis
```

OpenFresco: ESFourActuators3D.cpp

```
ESFourActuators3d::ESFourActuators3d(int tag,
    double actLength1, double actLength2, double actLength3, double actLength4,
    double rigidLength1, double rigidLength2, double rigidLength3, double rigidLength4,
    double rigidHeight, double rigidHeight1, double rigidHeight2,
    double rigidLinkN, double rigidLinkS, double rodLengthN, double rodLengthS,
    ExperimentalControl* control,
    int nlgeom, double philocx)
    : ExperimentalSetup(tag, control),
    L1(actLength1), L2(actLength2), L3(actLength3), L4(actLength4),
    a1(rigidLength1), a2(rigidLength2), a3(rigidLength3), a4(rigidLength4),
    h(rigidHeight), h1(rigidHeight1), h2(rigidHeight2),
    arlN(rigidLinkN), arlS(rigidLinkS), LrodN(rodLengthN), LrodS(rodLengthS),
    nlGeom(nlgeom), phiLocX(philocx), rotLocX(4,4), D(4), d(4)
{

    uxbeam = 0.0;
    uybeam = 0.0;
    upS = 0.0;
    upN = 0.0;
    theta1 = 0.0;
    theta2 = 0.0;
    theta3 = 0.0;
    theta4 = 0.0;
    theta11 = 0.0;
    theta22 = 0.0;
    theta33 = 0.0;
    theta44 = 0.0;

    // call setup method
    this->setup();

    for (int i=0; i<3; i++)
        firstWarning[i] = true;
}
```

OpenFresco: ESFourActuators3D.cpp

```
ESFourActuators3d::ESFour
    double actLength1, do
    double rigidLength1, d
    double rigidHeight, d
    double rigidLinkN, do
    ExperimentalControl* e
    int nlgeom, double phi
    : ExperimentalSetup(te
    L1(actLength1), L2(act
    a1(rigidLength1), a2(r
    h(rigidHeight), h1(rig
    arlN(rigidLinkN), arlS
    nlGeom(nlgeom), phiLoc
{

    uxbeam = 0.0;
    uybeam = 0.0;
    upS = 0.0;
    upN = 0.0;
    theta1 = 0.0;
    theta2 = 0.0;
    theta3 = 0.0;
    theta4 = 0.0;
    theta11 = 0.0;
    theta22 = 0.0;
    theta33 = 0.0;
    theta44 = 0.0;

    // call setup method
    this->setup();

    for (int i=0; i<3; i++
        firstWarning[i] =
}
```

```
int ESFourActuators3d::setSize(ID sizeT, ID sizeO)
{
    // check sizeTrial and sizeOut
    // for ESFourActuators3d object

    // a component of sizeT/sizeO must be equal
    // to 6 if it is non-zero.
    for (int i=0; i<OF_Resp_Time; i++) {
        if ((sizeT[i] != 0 && sizeT[i] != 6) ||
            (sizeO[i] != 0 && sizeO[i] != 6)) {
            opserr << "ESFourActuators3d::setSize - wrong sizeTrial/Out\n";
            opserr << "see User Manual.\n";
            opserr << "sizeT = " << sizeT;
            opserr << "sizeO = " << sizeO;
            return OF_ReturnType_failed;
        }
    }
    if ((sizeT[OF_Resp_Time] != 0 && sizeT[OF_Resp_Time] != 1) ||
        (sizeO[OF_Resp_Time] != 0 && sizeO[OF_Resp_Time] != 1)) {
        opserr << "ESFourActuators3d::setSize - wrong sizeTrial/Out\n";
        opserr << "see User Manual.\n";
        opserr << "sizeT = " << sizeT;
        opserr << "sizeO = " << sizeO;
        return OF_ReturnType_failed;
    }

    return OF_ReturnType_completed;
}
```

OpenFresco: ESFourActuators3D.cpp

```
int ESFourActuators3d::setup()
{
    // setup for ctrl/daq vectors of ESFourActuators3d
    sizeCtrl->Zero();
    (*sizeCtrl)[OF_Resp_Disp]  = 4;
    (*sizeCtrl)[OF_Resp_Time]  = 1;

    sizeDaq->Zero();
    (*sizeDaq)[OF_Resp_Force]  = 6;
    (*sizeDaq)[OF_Resp_Time]   = 1;
```

OpenFresco: ESFourActuators3D.cpp

```
int ESFourActuators3d::transfTrialDisp(const Vector* disp)
{
    // transforme yz coordinate system to local xy system
    //Vector D(4);
    D(0)=-(*disp)(1)*(*cDispFact)(1);
    D(1)=-(*disp)(3)*(*cDispFact)(3);
    D(2)=-(*disp)(4)*(*cDispFact)(4);
    D(3)=-(*disp)(2)*(*cDispFact)(2);

    // transforme displacements (ux, uy, phix, phiy) to local coordinate system
    //Vector d(4);
    d = rotLocX*D;

    // rigid body displacements due to vector d
    double pi = acos(-1.0);
    double alpha = 0.0;
    double beta = 0.0;

    if (d(2)==0) {
        alpha = (d(3)/fabs(d(3)))*pi/2.0;
        beta = -(d(3)/fabs(d(3)))*d(3);
    }
    else {
        alpha = atan(d(3)/d(2));
        beta =-(d(2)/fabs(d(2)))*sqrt(pow(d(2),2.0)+pow(d(3),2.0));
    }
    double gama=-alpha;
```

OpenFresco: ESFourActuators3D.cpp

```
Matrix A(3,3), B(3,3), C(3,3), V(3,4), V1(3,1), V2(3,1);
    A.Zero();
    A(0,0)= cos(gama); A(0,1)=sin(gama);
    A(1,0)=-sin(gama); A(1,1)=cos(gama);
    A(2,2)= 1.0;

    B.Zero();
    B(0,0)= 1.0;
    B(1,1)= cos(beta); B(1,2)=sin(beta);
    B(2,1)=-sin(beta); B(2,2)=cos(beta);

    C.Zero();
    C(0,0)= cos(alpha); C(0,1)=sin(alpha);
    C(1,0)=-sin(alpha); C(1,1)=cos(alpha);
    C(2,2)= 1.0;

    V.Zero();
    V(0,0)=-a1; V(0,2)=-a3;
    V(1,1)=-a2; V(1,3)=-a4;
    V(2,2)=h; V(2,3)=h;

Matrix R(3,3), Vr(3,4), Vr1(3,1), Vr2(3,1);
R.Zero(); Vr.Zero(); Vr1.Zero(); Vr2.Zero();
R=A*B*C;
Vr1(2,0)=h1; Vr2(2,0)=h2;
Vr=R*V; Vr1=R*V1; Vr2=R*V2;

uxbeam = D(0)+Vr1(0,0)*cos(phiLocX/180.0*pi)-Vr1(1,0)*sin(phiLocX/180.0*pi);
uybeam = D(1)+Vr2(0,0)*sin(phiLocX/180.0*pi)+Vr2(1,0)*cos(phiLocX/180.0*pi);
upN    = uybeam-sin(-D(2))*arlN;
upS    = uybeam-sin(-D(2))*arlS;
```

OpenFresco: ESFourActuators3D.cpp

```
    // linear geometry
    if (nlGeom == 0 )  {
        opserr << "Linear geometry case is not implemented yet";
    }

    // nonlinear geometry
    else if (nlGeom == 1)  {
        // actuator 1
        double u11 = L1+d(0)+a1+Vr(0,0);
        double u12 = d(1)+Vr(1,0);
        double u13 = Vr(2,0);
        (*cDisp)(0) = sqrt(pow(u11,2.0)+pow(u12,2.0)+pow(u13,2.0))-L1;
        theta1  = phiLocX*pi/180.0+atan(u12/u11);
        theta11 = atan(u13/sqrt(pow(u11,2.0)+pow(u12,2.0)));
        // actuator 2
        double u21 = d(0)+Vr(0,1);
        double u22 = L2+d(1)+a2+Vr(1,1);
        double u23 = Vr(2,1);
        (*cDisp)(1) = sqrt(pow(u21,2.0)+pow(u22,2.0)+pow(u23,2.0))-L2;
        theta2  = phiLocX*pi/180.0+atan(u21/u22);
        theta22 = atan(u23/sqrt(pow(u21,2.0)+pow(u22,2.0)));
        // actuator 3
        double u31 = L3+d(0)+a3+Vr(0,2);
        double u32 = d(1)+Vr(1,2);
        double u33 = Vr(2,2)-h;
        (*cDisp)(2) = sqrt(pow(u31,2.0)+pow(u32,2.0)+pow(u33,2.0))-L3;
        theta3  = phiLocX*pi/180.0+atan(u32/u31);
        theta33 = atan(u33/sqrt(pow(u31,2.0)+pow(u32,2.0)));
        // actuator 4
        double u41 = d(0)+Vr(0,3);
        double u42 = L4+d(1)+a4+Vr(1,3);
        double u43 = Vr(2,3)-h;
        (*cDisp)(3) = sqrt(pow(u41,2.0)+pow(u42,2.0)+pow(u43,2.0))-L4;
        theta4  = phiLocX*pi/180.0+atan(u41/u42);
        theta44 = atan(u43/sqrt(pow(u41,2.0)+pow(u42,2.0)));
    }

    return OF_ReturnType_completed;
}
```

OpenFresco: ESFourActuators3D.cpp

```
int ESFourActuators3d::transfDaqForce(Vector* force)
{
    // linear geometry
    if (nlGeom == 0 )  {
        opserr << "WARNING ESFourActuators3d::transfDaqForce() - "
               << "Linear geometry case is not implemented yet\n";
    }
    // nonlinear geometry
    else if (nlGeom == 1 )  {

        //Lateral forces that come from the rods
        double deltaFxN = -(uxbeam/LrodN)*(-1.0)*(*dForce)(0)*cos(D(2));
        double deltaFxS = -(uxbeam/LrodS)*(-1.0)*(*dForce)(1)*cos(D(2));
        double deltaFx  = deltaFxN+deltaFxS;

        double deltaFyN = (-1.0)*(*dForce)(0)*sin(D(2));
        double deltaFyS = (-1.0)*(*dForce)(1)*sin(D(2));
        double deltaFy  = deltaFyN+deltaFyS;

        //Moments that come from the rods
        double FzN = (*dForce)(0)*cos(-D(2));
        double FzS = (*dForce)(1)*cos(-D(2));
        double Fz  = FzN+FzS;
        double MyV = -Fz*(uxbeam-D(0));
        double MyR = MyV+deltaFx*h1;
        double MxR = Fz*(uybeam+0.5*Hbeam*sin(-D(2))-D(1))-deltaFy*(h2+0.5*Hbeam);
```

OpenFresco: ESFourActuators3D.cpp

```
        // Lateral forces that come from actuators
        double F1x = (*dForce)(2)*cos(theta1)*cos(theta11);
        double F1y = (*dForce)(2)*sin(theta1)*cos(theta11);
        double F2x = -(*dForce)(3)*cos(theta2)*cos(theta22);
        double F2y = (*dForce)(3)*sin(theta2)*cos(theta22);
        double F3x = (*dForce)(4)*cos(theta3)*cos(theta33);
        double F3y = (*dForce)(4)*sin(theta3)*cos(theta33);
        double F3z = (*dForce)(4)*sin(theta33);
        double F4x = -(*dForce)(5)*cos(theta4)*cos(theta44);
        double F4y = (*dForce)(5)*sin(theta4)*cos(theta44);
        double F4z = (*dForce)(5)*sin(theta44);

        //Moments that come from actuators
        double MxA = -(F3y*ha3+F4y*ha4)+F3z*ay3+F4z*ay4;
        double MyA = (F3x*ha3+F4x*ha4)-(F3z*ax3+F4z*ax4);

        // Total lateral forces and moments
        double Fx = F1x+F2x+F3x+F4x+deltaFx;
        double Fy = F1y+F2y+F3y+F4y+deltaFy;
        double Mx = MxA+MxR;
        double My = MyA+MyR;

        (*force)(0) = 0;
        (*force)(1) = -Fx*(*dForceFact)(1);
        (*force)(2) = -My*(*dForceFact)(2);
        (*force)(3) = -Fy*(*dForceFact)(3);
        (*force)(4) = -Mx*(*dForceFact)(4);
        (*force)(5) = 0;
    }
    return OF_ReturnType_completed;
}
```

Definition of ExpSetup in TCL Script

```
# -------------------------
# Define experimental setup
# -------------------------
# expSetup FourActuators $tag <-control ctrlTag> $L1 $L2 $L3 $L4 $a1 $a2 $a3 $a4
#                         $h $h1 $h2 $arlN $arlS $LrodN $LrodS <-nlGeom> <-phiLocX phi>
#                         <-ctrlDispFact f> <-ctrlVelFact f> <-ctrlAccelFact f>
#                         <-ctrlForceFact f> <-ctrlTimeFact f> <-daqDispFact f>
#                         <-daqVelFact f> <-daqAccelFact f> <-daqForceFact f> <-daqTimeFact f>
expSetup FourActuators 1  -control 1  97.8125  96.375  98.5  97.5  18.1875 18.1875  17.625  17.625 \
                        18.375  31.5  35.8125  7.375  7.4375  85 84.9375  -nlGeom
                        -ctrlDispFact 8.398950187  8.398950187 1.0 1.0 \
                        -daqForceFact 97.73925293 97.73925293 11.6370798 97.73925293 11.6370798 1.0
```

Units in analytical model: kN, m

Units in experimental setup: kips, in

Scaling factors:

$S_d = 4.6875$

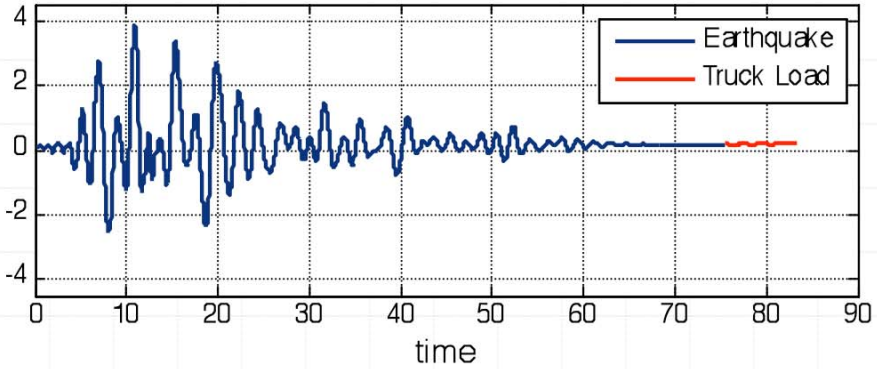
$S_f = 1/S_d^2$

$S_m = 1/S_d^3$

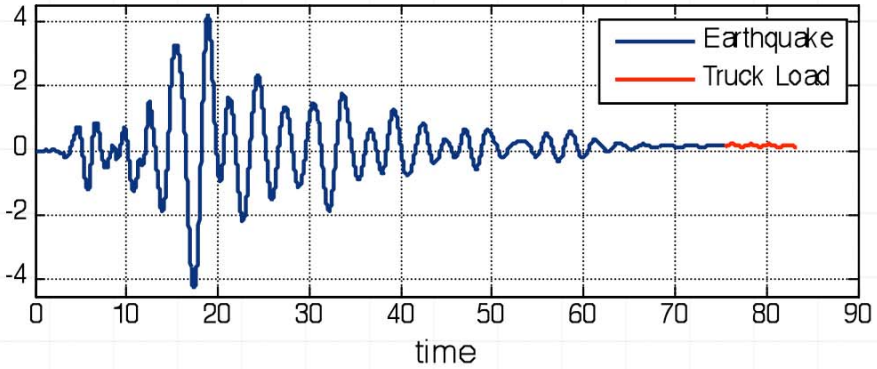
Results: Displacements



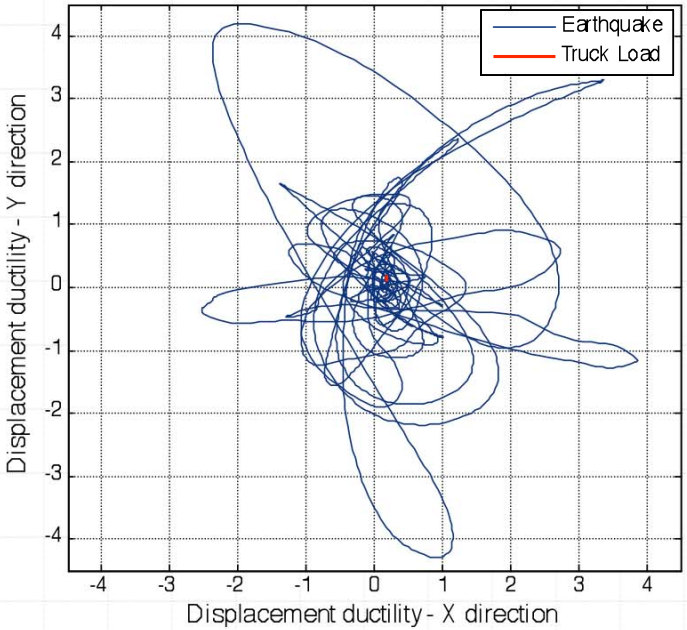
Normalized displacement at the top of the column 1 - X direction



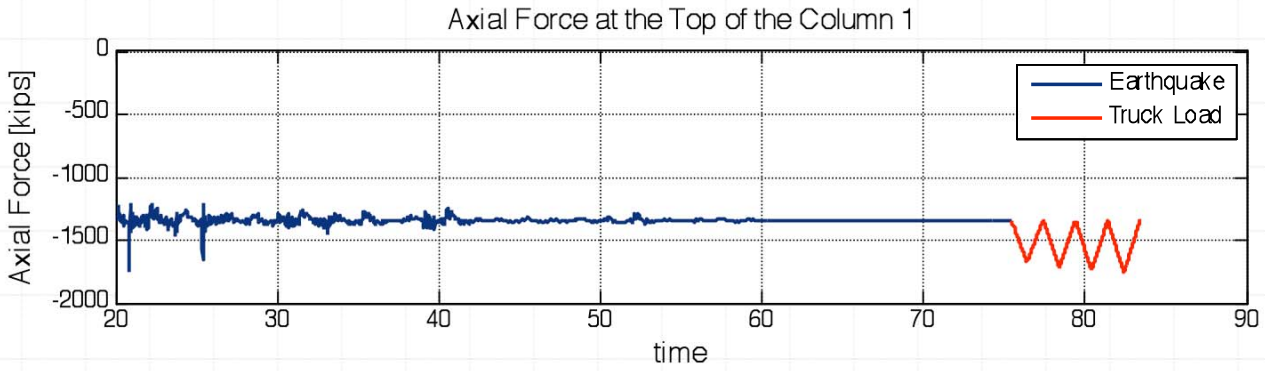
Normalized displacement at the top of the column 1 - Y direction

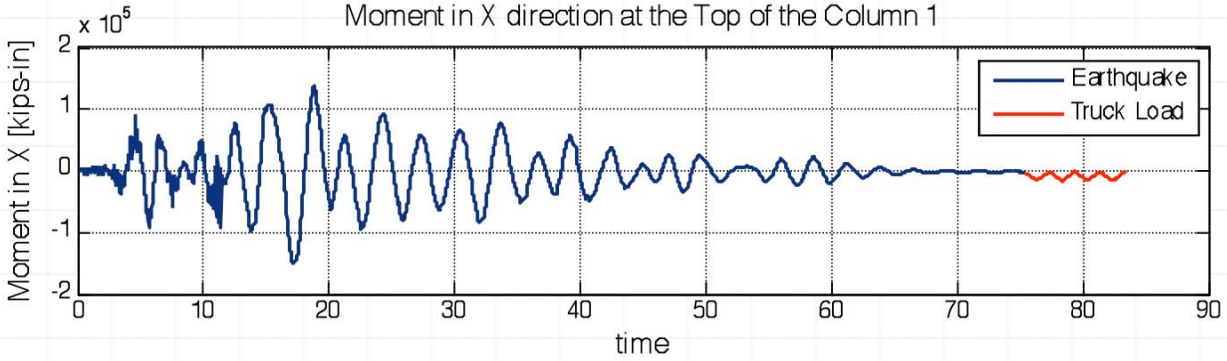


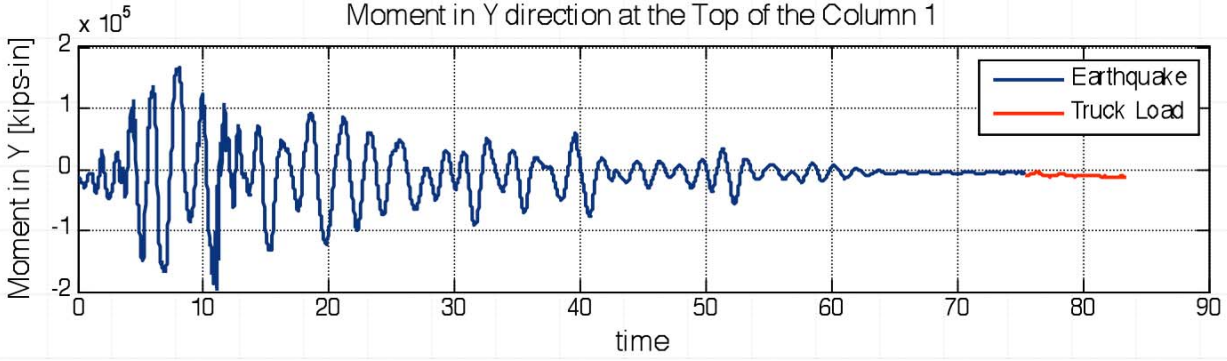
Displacement orbit at the top of the column 1



Results: Forces







Four truck positions on a bridge

Questions?
Thank you!

http://openfresco.neesforge.nees.org

The development of OpenFresco has been sponsored in parts by the National Science Foundation through grants from the NEES Consortium, Inc.

Department of Civil and Environmental Engineering
University of California, Berkeley

